FAZENDA AURORA · APOSTILA DO ALUNO

UC 03

Tinkercad: do código ao circuito

Simule sensores, sinais e uma irrigação inteligente antes de usar componentes reais.

Aprender para transformar

Você chegou a uma fazenda que precisa de organização, decisões e automação. Cada capítulo transforma uma necessidade da Aurora em uma habilidade prática. Leia, altere os exemplos, teste e explique o seu raciocínio.

Como usar esta apostila

1. Leia a situação e os conceitos. 2. Reproduza o exemplo no ambiente indicado. 3. Faça a missão sem apenas copiar a resposta. 4. Teste situações comuns e limites. 5. Use o checkpoint para revisar e a mentora para discutir dúvidas.

Nesta UC

01 Do algoritmo à bancada virtual

02 Primeiro circuito: o LED do celeiro

03 Tempo e sinais: o aviso de manutenção

04 Botão: o portão sob controle

05 Leitura analógica: a umidade simulada

06 PWM: ajustar a intensidade

07 Monitoramento: uma decisão que podemos enxergar

08 Missão final: irrigação com duas condições

Os exemplos são didáticos. A avaliação da IA é formativa e deve ser confrontada com testes. Os laboratórios do navegador não substituem um compilador C++, uma VM ou a montagem no Tinkercad.

CAPÍTULO 01 · 15 MIN DE LEITURA E PRÁTICA

Do algoritmo à bancada virtual

A lógica já existe. Agora precisamos ver como ela conversa com componentes e sinais.

Entenda o conceito

Tinkercad Circuits permite montar e simular circuitos no navegador. Crie uma conta conforme as regras do serviço, entre em Circuits e comece um circuito novo. Você fará os projetos reais lá; o laboratório da Aurora treina a lógica de sensores.

Uma entrada informa um estado ao programa; uma saída atua no circuito. O Arduino Uno possui pinos digitais e entradas analógicas. GND é a referência elétrica compartilhada entre componentes.

No primeiro projeto, uma luz representará a bomba. Um motor real não deve ser ligado diretamente ao pino do Arduino: ele exige um circuito de acionamento e alimentação adequados.

Exemplo para experimentar

```
// Estrutura minima de um sketch Arduino
void setup() {
    // Configuracao inicial
}

void loop() {
    // Rotina que se repete
}
```

Sua missão na Aurora

Crie um circuito no Tinkercad com Arduino Uno. Descreva o papel de setup, loop, entrada, saída e GND. Explique por que usaremos um LED para representar a bomba.

Antes de enviar

- Sketch compreendido
- Entradas e saídas distinguidas
- Representação segura do atuador

Atenção: Circuito virtual e circuito físico são diferentes. Não copie um acionamento de motor sem dimensionar os componentes.

Checkpoint

Qual função se repete no sketch Arduino?

A) setup

B) loop

C) main do nosso exemplo C++

CAPÍTULO 02 · 15 MIN DE LEITURA E PRÁTICA

Primeiro circuito: o LED do celeiro

Quando o celeiro está em operação, uma luz deve avisar a equipe.

Entenda o conceito

Um LED tem polaridade. No circuito inicial, ligue o pino digital 9 a um resistor de 220 Ω; depois ao ânodo do LED. Ligue o cátodo ao GND. O resistor limita a corrente.

pinMode define como um pino será usado. digitalWrite coloca uma saída digital em HIGH ou LOW. Escolha constantes para nomes de pinos, evitando números espalhados pelo programa.

Antes de simular, confira as conexões. Se o LED não acender, verifique polaridade, resistor, GND e o pino usado no código. O objetivo é observar uma relação entre comando e sinal.

Exemplo para experimentar

```
const int LED = 9;

void setup() {
    pinMode(LED, OUTPUT);
    digitalWrite(LED, HIGH);
}

void loop() {
}
```

Sua missão na Aurora

Monte Arduino, resistor de 220 Ω e LED no Tinkercad. Programe o pino 9 para manter o LED ligado. Entregue o código e descreva as conexões.

Antes de enviar

- Resistor em série
- Polaridade e GND corretos
- Pino coerente com o código

Atenção: Não retire o resistor só porque a simulação parece funcionar. O circuito físico precisa de limitação de corrente.

Checkpoint

Qual componente limita a corrente do LED neste circuito?

A) Resistor

B) O texto do programa

C) Um segundo LED

CAPÍTULO 03 · 15 MIN DE LEITURA E PRÁTICA

Tempo e sinais: o aviso de manutenção

Um aviso intermitente é mais fácil de perceber do que uma luz sempre acesa.

Entenda o conceito

delay recebe um tempo em milissegundos. delay(1000) espera aproximadamente um segundo. Durante essa espera, seu sketch não executa as próximas instruções da mesma rotina.

Para um pisca-pisca simples, alternar HIGH e LOW com pausas é suficiente. Para ler vários sensores continuamente, essa abordagem pode atrasar respostas; millis permite criar temporizações sem uma espera bloqueante longa.

Comece observando o comportamento. A frequência de um sinal depende de todo o ciclo, não de apenas uma das pausas. Uma pausa ligada de 500 ms e outra desligada de 500 ms formam um ciclo de aproximadamente um segundo.

Exemplo para experimentar

```
const int LED = 9;
void setup() { pinMode(LED, OUTPUT); }
void loop() {
    digitalWrite(LED, HIGH);
    delay(500);
    digitalWrite(LED, LOW);
    delay(500);
}
```

Sua missão na Aurora

Faça o LED ficar ligado por 500 ms e desligado por 500 ms. Explique a duração do ciclo e por que delay pode ser um problema em um sistema com vários sensores.

Antes de enviar

• Alternância correta

• Unidade de tempo compreendida

• Limitação de delay explicada

Atenção: Não diga que delay executa outras tarefas em paralelo. A próxima instrução da rotina espera o tempo terminar.

Checkpoint

Quantos milissegundos correspondem a um segundo?

A) 10

B) 100

C) 1000

CAPÍTULO 04 · 15 MIN DE LEITURA E PRÁTICA

Botão: o portão sob controle

A equipe precisa de um botão para avisar quando deseja abrir o portão.

Entenda o conceito

Uma entrada digital lê HIGH ou LOW. INPUT_PULLUP ativa uma resistência interna de pull-up, evitando que a entrada fique sem uma referência definida.

Neste exercício, ligue o botão entre o pino 2 e GND. Com INPUT_PULLUP, solto corresponde a HIGH; pressionado corresponde a LOW. Esta inversão precisa aparecer na sua condição.

Botões mecânicos podem gerar pequenas oscilações ao trocar de estado, chamadas de bouncing. Para sistemas que contam cliques, é importante aplicar uma estratégia de debounce. Nesta primeira prática, vamos apenas acompanhar o estado.

Exemplo para experimentar

```
const int BOTAO = 2;
const int LED = 9;
void setup() {
    pinMode(BOTAO, INPUT_PULLUP);
    pinMode(LED, OUTPUT);
}
void loop() {
    bool pressionado = digitalRead(BOTAO) == LOW;
    digitalWrite(LED, pressionado ? HIGH : LOW);
}
```

Sua missão na Aurora

Monte um botão entre o pino 2 e GND, usando INPUT_PULLUP. O LED do pino 9 deve acender apenas quando o botão for pressionado. Explique por que LOW significa pressionado.

Antes de enviar

• INPUT_PULLUP usado

• Botão entre pino e GND

• Condição LOW explicada

Atenção: Não use a mesma condição de um circuito com pull-down sem conferir o esquema.

Checkpoint

Com este circuito INPUT_PULLUP, a leitura ao pressionar é:

A) HIGH

B) LOW

C) 1023

CAPÍTULO 05 · 15 MIN DE LEITURA E PRÁTICA

Leitura analógica: a umidade simulada

A fazenda precisa medir variações, não apenas estados ligados e desligados.

Entenda o conceito

No Arduino Uno, analogRead retorna uma leitura de 0 a 1023 para sua entrada analógica. Um potenciômetro pode representar uma variável no laboratório, como a umidade.

Ligue as extremidades do potenciômetro a 5 V e GND e o terminal central a A0. O valor depende da tensão nesse terminal. Use o monitor serial para observar a leitura.

Uma leitura bruta não é automaticamente uma porcentagem de umidade real. Um sensor físico precisa ser calibrado. Neste treino, convertemos a posição do potenciômetro para 0 a 100 somente como um modelo didático.

Exemplo para experimentar

```
void setup() { Serial.begin(9600); }
void loop() {
    int bruto = analogRead(A0);
    int umidade = map(bruto, 0, 1023, 0, 100);
    Serial.print("Umidade simulada: ");
    Serial.println(umidade);
    delay(200);
}
```

Sua missão na Aurora

Monte um potenciômetro em A0. Exiba a leitura bruta e a porcentagem simulada no monitor serial. Teste três posições e explique por que um sensor real exigiria calibração.

Antes de enviar

- Ligação de A0 correta
- Leitura e conversão distinguidas
- Três posições testadas

Atenção: A orientação de um sensor real pode ser diferente da do potenciômetro usado na simulação.

Checkpoint

Qual é a faixa da leitura analogRead no Uno deste exercício?

A) 0 a 100

B) 0 a 1023

C) Somente HIGH e LOW

CAPÍTULO 06 · 15 MIN DE LEITURA E PRÁTICA

PWM: ajustar a intensidade

A estufa não precisa de uma única intensidade de luz em todas as situações.

Entenda o conceito

analogWrite, nos pinos PWM do Uno, recebe um valor entre 0 e 255. Ele altera o ciclo de trabalho de pulsos, não transforma qualquer pino em uma saída analógica contínua.

Use um pino que suporte PWM, como o 9. Um potenciômetro pode controlar a intensidade de um LED com resistor. map converte a faixa 0–1023 para 0–255.

No laboratório visual, observe como o comando de intensidade muda. Para projetos físicos, um atuador com maior consumo continua exigindo um circuito de acionamento adequado.

Exemplo para experimentar

```
const int LED = 9;
void setup() { pinMode(LED, OUTPUT); }
void loop() {
    int leitura = analogRead(A0);
    int intensidade = map(leitura, 0, 1023, 0, 255);
    analogWrite(LED, intensidade);
}
```

Sua missão na Aurora

Controle a intensidade do LED no pino 9 com um potenciômetro em A0. Explique a diferença entre analogRead e analogWrite e registre três intensidades.

Antes de enviar

- Pino PWM escolhido
- Conversão de faixa adequada
- Leitura e escrita explicadas

Atenção: Nem todos os pinos do Uno suportam PWM. Confira o pino antes de atribuir o problema ao código.

Checkpoint

Qual faixa usamos em analogWrite no Uno?

A) 0 a 1

B) 0 a 255

C) 0 a 1023

CAPÍTULO 07 · 15 MIN DE LEITURA E PRÁTICA

Monitoramento: uma decisão que podemos enxergar

A bomba não pode ligar sem que a equipe saiba o motivo da decisão.

Entenda o conceito

Combine uma entrada analógica com uma condição e uma saída digital. Mostre no monitor serial a leitura e o estado da saída para facilitar a depuração.

Abaixo de 40% de umidade simulada, ligue o LED que representa a bomba; em 40% ou acima, desligue. Teste perto do limite. Uma leitura ruidosa perto do limiar pode causar alternância frequente.

Histerese usa limites diferentes para ligar e desligar, reduzindo essa alternância. Ela é uma melhoria possível após dominar a regra simples deste capítulo, não algo que o circuito básico já faz sozinho.

Exemplo para experimentar

```
const int BOMBA = 9;
void setup() {
    pinMode(BOMBA, OUTPUT);
    Serial.begin(9600);
}
void loop() {
    int umidade = map(analogRead(A0), 0, 1023, 0, 100);
    bool irrigar = umidade < 40;
    digitalWrite(BOMBA, irrigar ? HIGH : LOW);
    Serial.print(umidade);
    Serial.println(irrigar ? " - IRRIGAR" : " - AGUARDAR");
    delay(200);
}
```

Sua missão na Aurora

Crie o controle de irrigação com um potenciômetro e LED. Mostre valores e decisão no monitor serial. Teste 39%, 40% e 41% na lógica do laboratório e explique a fronteira.

Antes de enviar

- Regra do limiar correta
- Saída e relatório serial coerentes
- Testes da fronteira

Atenção: Não diga que o LED é uma bomba real. Ele representa o comando ao atuador na simulação.

Checkpoint

Com a regra umidade < 40, o LED deve acender em:

A) 39%

B) 40%

C) 41%

CAPÍTULO 08 · 25 MIN DE LEITURA E PRÁTICA

Missão final: irrigação com duas condições

A Fazenda Aurora agora pode combinar umidade, água disponível e uma saída de irrigação.

Entenda o conceito

Use dois potenciômetros: A0 representa a umidade e A1 representa o reservatório. Ambos são modelos didáticos de 0 a 100%. O LED no pino 9 representa o comando da bomba.

A regra é a mesma da UC 1: irrigar quando a umidade for menor que 40 e o nível do reservatório for pelo menos 20. Uma função torna essa regra fácil de reconhecer e testar.

Entregue o link do projeto Tinkercad, o código, a descrição das conexões e uma tabela de testes. A mentora avalia o texto e o código enviados; não inspeciona automaticamente o circuito do link. Uma montagem física exige revisão dos componentes de acionamento.

Exemplo para experimentar

```
const int BOMBA = 9;
bool deveIrrigar(int umidade, int nivel) {
    return umidade < 40 && nivel >= 20;
}
void setup() {
    pinMode(BOMBA, OUTPUT);
    Serial.begin(9600);
}
void loop() {
    int umidade = map(analogRead(A0), 0, 1023, 0, 100);
    int nivel = map(analogRead(A1), 0, 1023, 0, 100);
    bool irrigar = deveIrrigar(umidade, nivel);
    digitalWrite(BOMBA, irrigar ? HIGH : LOW);
    Serial.print("Umidade: "); Serial.print(umidade);
    Serial.print(" Nivel: "); Serial.print(nivel);
    Serial.println(irrigar ? " IRRIGAR" : " AGUARDAR");
    delay(200);
}
```

Sua missão na Aurora

Construa a irrigação com dois potenciômetros, Arduino e LED com resistor. Envie código, link, descrição das conexões e os testes (39,20), (40,20), (39,19) e (39,80). Explique como as três UCs se conectam.

Antes de enviar

- Conexões descritas com precisão
- Regra com duas condições
- Quatro casos de teste
- Integração C++, Linux e circuito explicada

Atenção: Não alimente um motor diretamente por um pino do Arduino. O LED é o atuador didático desta missão.

Checkpoint

Qual situação deve irrigar?

A) Umidade 39 e reservatório 19

B) Umidade 40 e reservatório 80

C) Umidade 39 e reservatório 20

Avaliação e testes

A nota formativa vai de 0 a 100. Não é um certificado nem uma avaliação oficial da escola. O mais importante é usar o retorno para revisar a solução e demonstrar que ela funciona.

Solução do problema: até 40 pontos

Lógica e domínio dos conceitos: até 25 pontos

Testes e casos de borda: até 20 pontos

Clareza e explicação: até 15 pontos

Modelo de tabela de testes

Para cada teste, registre: entrada ou situação; resultado esperado; resultado observado; conclusão. Um caso que falhou é uma informação útil para melhorar, não algo a esconder.

Como conversar com a mentora

Diga em qual missão está, cole um trecho pequeno e informe o erro ou a saída observada. Peça uma pista antes de pedir a solução completa. Não envie senhas, tokens ou dados pessoais de outras pessoas.

Gabarito comentado dos checkpoints

01. B — No ambiente Arduino, loop é chamado repetidamente após setup.

02. A — O resistor em série limita a corrente, protegendo o componente.

03. C — Um segundo contém 1000 milissegundos.

04. B — O botão conecta o pino ao GND, produzindo LOW.

05. B — O conversor analógico-digital do Uno tem resolução de 10 bits, com 1024 valores possíveis.

06. B — No Uno, analogWrite trabalha com uma faixa de 0 a 255 para definir o ciclo de trabalho PWM.

07. A — Só 39 está abaixo do limite. Em 40 e 41, o sistema aguarda.

08. C — A umidade está abaixo de 40 e o reservatório está no mínimo necessário de 20.

Referências e ferramentas oficiais

GNU GCC · compilador C++

Ubuntu · VM com VirtualBox

VirtualBox · download oficial

Ubuntu 24.04 LTS · imagens oficiais

Arduino · referência da linguagem

Tinkercad Circuits